

ProsaBuddy: Assisting Mechanized Real-Time Schedulability Analysis with LLM-based Agents

Abstract—Rigorous schedulability analysis is essential for hard real-time systems, yet errors in pen-and-paper proofs threaten the safety of critical applications. The Prosa initiative addresses this by offering a foundation for building machine-checkable schedulability analysis proofs in the Rocq proof assistant. However, the substantial time and expertise required to construct such proofs remain a major barrier for wider adoption of Prosa. In this work, we present ProsaBuddy, an LLM-based agent system designed to lower the effort needed to develop mechanized real-time schedulability proofs. ProsaBuddy employs an intensive ReAct loop with retrieval over the Prosa codebase, direct access to Rocq tools, and optional human-written proof hints. It employs a subgoal-delegation architecture, decomposing a target lemma into independent subgoals and dispatches them to sub-agents for proof. We evaluate ProsaBuddy on a mini benchmark drawn from the real-time scheduling literature, without overlapping with existing Prosa library. Experiment results show that ProsaBuddy significantly outperforms state-of-the-art LLM-based Rocq automated proving agent systems and the general coding agent OpenCode. Ablation studies show that the gains stem from the combination of several aspects of our design choices. Overall, ProsaBuddy can successfully prove many non-trivial lemmas, including those with proofs of hundreds of tactics. Yet, ProsaBuddy is intended not as a fully automated replacement for human researchers but as an assistant that handles tedious low-level proof details, allowing researchers to focus on high-level proof ideas and guide the system to efficient proof completion.

I. INTRODUCTION

In hard real-time systems, rigorous real-time performance analysis is essential to guarantee the system’s timing behavior to consistently meet specified constraints. Researchers in this field typically develop schedulability analysis theorems that provide efficient methods for evaluating real-time performance. The development of such analyses has traditionally relied on human effort and has been conducted in a pen-and-paper manner. However, numerous recent reports have revealed errors in schedulability analyses documented in literatures [1]–[4]. Since real-time systems underpin safety-critical applications, such analysis errors can lead to disastrous consequences.

Prosa [5] is an initiative that aims to fundamentally address this problem. Prosa provides a repository of definitions and proofs for mechanized (i.e., machine-checkable) real-time schedulability analysis. It is built and verified using Rocq [6], a mature interactive theorem prover with support for higher-order logic and computation. Prosa builds dedicated infrastructure and methodological frameworks to enable detailed, reliable, yet readable real-time schedulability analysis. Widespread adoption of the Prosa approach within the real-time systems research community would significantly elevate

the rigor, credibility, and foundational robustness of theoretical developments in the field.

Despite its potential, a practical barrier prevents Prosa from achieving broader adoption within the research community: developing proofs in Rocq is very time-consuming. Although Prosa has several features that enhance its accessibility to real-time systems researchers, such as prioritizing readability and adhering to established proof conventions within the community, the overhead of proof development remains substantial. According to the experience of our own research group, proving lemmas in a real-time scheduling paper may require several weeks, even for those with moderate familiarity with Prosa and Rocq. For newcomers without prior experience, the time investment is even more demanding. This substantial effort is likely a primary factor discouraging researchers from adopting the Prosa approach in their own work.

In this work, we present ProsaBuddy, an LLM-based agent system designed to lower the effort barrier to adopting the Prosa approach for developing mechanized schedulability proofs. While LLMs have demonstrated impressive automated theorem proving capabilities in domains such as pure mathematics, their application to theorem proving in real-time scheduling remains entirely unexplored. Although real-time scheduling lemmas can broadly be regarded as mathematical problems, they exhibit unique features that make them significantly more challenging to formalize and prove automatically than the pure-math problems in standard automated theorem proving benchmarks, as will be discussed in Section III-A.

We build ProsaBuddy as an agent system that heavily relies on inference-time scaling through an intensive ReAct (Reasoning & Acting) loop [7]. It combines retrieval over the Prosa codebase to supply necessary definitions and proof patterns, direct access to various Rocq tools via dedicated skills to obtain rich feedback within the loop, and optional human-written proof hints that provide high-level guidance. ProsaBuddy adopts a subgoal-delegation architecture: a primary agent decomposes the target lemma into smaller, independent subgoals and dispatches them to specialized sub-agents, each receiving a structured task bundle that includes the subgoal, context, and a proof state snapshot. We enhance the agents with decomposition guidelines and domain-specific proving skills, improving their ability to navigate the deep proof chains typical of real-time schedulability analysis.

We conducted experiments to evaluate the performance of ProsaBuddy, comparing it with state-of-the-art LLM-based Rocq automated theorem proving systems, including PALM [8], COPRA [9], and RocqStar [10], as well as the

general-purpose coding agent OpenCode [11], which serves as a broad baseline. The comparison is conducted on a mini-benchmark of 24 lemmas that we built from the real-time scheduling literature; none of these lemmas appear in the existing Prosa library. Experimental results show that ProsaBuddy significantly outperforms all competing systems. We further extend the evaluation to a larger set of lemmas taken directly from the Prosa library, and the results confirm the superiority of ProsaBuddy. Ablation studies reveal that these gains arise from the combined effect of several design choices in ProsaBuddy. Additionally, we assess the impact of different backbone LLMs on the performance of ProsaBuddy.

Overall, ProsaBuddy can successfully prove many non-trivial schedulability analysis lemmas, including some whose proofs require hundreds of tactics. Nevertheless, we do not claim that ProsaBuddy can replace human researchers in independently proving schedulability analysis lemmas. Instead, ProsaBuddy mainly aims to liberate researchers from tedious low-level details in lengthy Rocq proofs, saving them time so they can devote more effort to developing high-level proof ideas, which can then guide ProsaBuddy to complete the proofs efficiently.

All developed prompts, skills, benchmark lemmas, generated proofs, and traces of the proving procedure are provided in the supplementary materials accompanying this submission.

II. BACKGROUND AND RELATED WORK

A. Rocq

Rocq (formerly known as Coq) is a proof assistant based on dependent type theory, allowing users to write mathematical definitions, executable algorithms, and prove properties about them interactively. Following the project’s renaming, we will refer to Rocq exclusively when introducing ProsaBuddy. One key difference between human-written and Rocq proofs is the level of detail: a simple intuition often requires many explicit steps. For example, in the example of Figure 1, the lemma states that a job scheduled at time t cannot be completed by t . For humans, this is trivial. In Rocq, however, the proof needs many steps. So developing Rocq proofs is effort-demanding.

Rocq does not draw a fundamental distinction between Lemma, Theorem, Fact, or similar keywords – they all introduce a conclusion to be proved. In this paper, we will use the term *lemma* uniformly for any such goal to prove, regardless of which keyword was used in its declaration. A lemma statement in Rocq typically introduces variables, definitions, hypotheses, and a lemma statement. The proof is enclosed by `Proof.` and `Qed.`, and the user guides the proof by applying *tactics*, which are commands that transform the current proof state (e.g., those marked in blue in Figure 1). Rocq provides a rich set of tactics, ranging from basic, primitive tactics that manipulate the proof state directly (e.g., `move`, `rewrite`, `apply`, `exact`) to powerful automation tactics that can search for proofs or solve decidable fragments automatically (e.g., `auto`, `eauto`, `lia`, `ring`).

In the presentation of this paper, we deliberately minimize the reader’s dependency on understanding the syntax and

```

1 Context {Job: eqType}.
2 Variable job_arrival: Job → time.
3 Variable job_cost: Job → time.
4 Variable sched: schedule Job.
5 Context {Task: eqType}.
6 Variable job_task: Job → Task.
7 Definition scheduled_at (j: Job) (t: time) := sched t == Some j.
8 Definition service (j: Job) (t: time) :=
9   \sum_(0 <= t' < t) scheduled_at j t'.
10 Definition completed_by (j: Job) (t: time) :=
11   job_cost j <= service j t.
12 Hypothesis H_service_leq_job_cost:
13   forall j t, service j t <= job_cost j.
14 Lemma scheduled_implies_not_completed:
15   forall j t, scheduled_at j t → ~ completed_by j t.
16 Proof.
17   move => t SCHED.
18   rewrite /completed_by. move=> H_sched. rewrite -ltnNge.
19   have H_comp := H_service_leq_job_cost t (SCHED + 1).
20   rewrite /service in H_comp. rewrite addn1 in H_comp.
21   rewrite big_nat_recr //:= in H_comp.
22   rewrite H_sched /= in H_comp. rewrite /service.
23   by rewrite addn1 in H_comp.
24 Qed.

```

Fig. 1. Example of a schedulability analysis lemma in Rocq

semantics of Rocq. Our goal is to let readers who are not familiar with Rocq still grasp the key ideas, even if they skip the details of the examples. For some important examples that are key to understanding our methodology, we will introduce the meaning of the tactics locally. For a comprehensive introduction to all tactics mentioned in the examples of this paper, we refer the reader to the Rocq reference manual [12].

B. Prosa

Prosa [5] is a Rocq-based foundation for verified real-time scheduling theory. It emphasizes readable specifications, explicit assumptions, and proof reuse. Prosa’s specifications are structured in the SSReflect proof style [13], which makes them both human-readable and machine-checkable. This combination of clarity and rigor also makes Prosa well-suited for LLM-based proof automation, as its good readability is informative not only to human users but also to LLM. Prosa has been extended to verified scheduling infrastructure in several directions in follow-up work, e.g., sustainability theory and weak sustainability [14], Abstract Response-Time Analysis (ARTA) based on the busy-window principle [15], FIFO response-time analysis with arbitrary deadlines and arrival curves [16], and connections between response-time analysis and network calculus [17]. Furthermore, Prosa serves as the certificate generator for POET [18], the first foundational response-time analysis tool that produces fully machine-checkable Coq/Rocq certificates. Another recent work integrates Prosa with RefinedC [19], connecting Prosa’s verified response-time analysis with C implementation verification, thereby enabling end-to-end certification of interrupt-free real-time schedulers implemented in low-level code. Together, these works make Prosa a useful infrastructure for verified real-time systems. They provide a common formal language for scheduling models, assumptions, and response-time theorems, and they support proof reuse across scheduling policies and analysis styles. However, Prosa still covers only part of real-time scheduling theory.

TABLE I
LLM-BASED AUTOMATED COQ/ROCQ THEOREM PROVING

Name	Source	Year	Prover	Availability	Open-source
CoqPilot	[35]	2024	Coq	Yes	Yes
PALM	[8]	2024	Coq	Yes	Yes
COPRA	[9]	2024	Coq	Yes	Yes
RocqStar	[10]	2025	Rocq	Yes	Yes
Rango	[36]	2025	Coq	Yes	Yes
ReCent-Prover	[37]	2026	Rocq	No	No

C. Automated Theorem Proving

Mechanized proofs are inherently hard to construct. Before the emergence of LLMs, many efforts sought to improve automation for different theorem provers, for instance, via powerful tactics that search for proofs or solve decidable fragments automatically. For Coq/Rocq, SSReflect [13], Mathematical Components [20], and CoqEAL [21] enable reuse of recurring proof patterns through reflection and proof-oriented computation; SMTCoq [22], smt4coq [23], and CoqHammer [24] connect Rocq goals to external solvers or proof reconstruction; Ltac/Ltac2 [25], Mtac/Mtac2 [26], [27], Coq-Elpi [28], and MetaCoq [29] support domain-specific tactics and reusable proof infrastructure. ML4PG [30], CoqGym [31], Tactician [32], Proverbot9001 [33], TacTok [34], and ASTactic [31] explored proof-pattern mining, tactic prediction, local proof search, and proof-step prediction. These automation techniques substantially reduce routine proof effort, but they do not fully eliminate the high demand for expert intervention. Large mechanized developments still require experts to decompose complex goals, formulate auxiliary lemmas, choose suitable abstractions, and coordinate induction arguments.

LLMs have recently been used for automated theorem proving across several proof assistants. Existing work has been done for different theorem provers, including Coq/Rocq [8]–[10], [35]–[37], Lean [38]–[48], and Isabelle/HOL [49]–[51]. In the following, we focus on the work for Coq/Rocq, and refer to recent survey papers [52], [53] for a more comprehensive overview of the field. Table lists the LLM-based automated theorem proving systems for Rocq. CoqPilot [35] provides a Coq-oriented assistant implemented as a VS Code extension, where users mark proof holes with `admit`, configure completion services, and invoke the tool inside an open Coq file to generate, check, and substitute candidate proofs through `coqlsp` during interactive editing. COPRA [9] formulates proof search as an in-context interactive process where the model proposes tactics and observes proof-state feedback. PALM [8] uses a generate-then-repair architecture to analyze and repair low-level proof errors. More recent systems combine proof generation with retrieval and multi-step interaction. Rango [36] retrieves relevant premises and similar proofs from the current Coq project. RocqStar [10] and ReCent-Prover [37] further introduce planning, reflection, and strategy-aware retrieval into Rocq proof generation.

Overall, this body of work shows a shift from isolated tactic prediction to systems that combine proof synthesis, retrieval,

prover feedback, repair, and agentic control. These systems can generate longer proof fragments, use local context, revise failed attempts, and sometimes use informal explanations or proof sketches as guidance. However, proofs in specialized domains such as real-time scheduling often rely on recurring semantic structures, including scheduling assumptions, workload and processor models, invariants, response-time bounds, rewriting conventions, and sequences of lemmas. Existing methods usually treat reusable knowledge as retrieved premises, similar proofs, or short-term proof-state contexts. They do not make domain-specific proof procedures explicit as reusable skills with applicability conditions, required assumptions, expected effects, and adaptation strategies. This leaves a gap between general LLM-based proof generation and domain-aware proof automation for Coq/Rocq libraries.

III. PROSABUDDY

A. Overview

Before presenting our approach, we first discuss the major differences between the problem tackled in this paper, automated theorem proving for real-time schedulability analysis, and the more commonly studied application of LLM-based automated theorem proving to pure mathematics, which raise new challenges to our work.

The theorem-proving capability of LLMs is obtained through pre-training on large-scale corpora such as Mathlib [54]. In contrast, public data for mechanized schedulability analysis is very limited. The Prosa library is the main available resource, but its volume is orders of magnitude smaller than the corpora for mathematics. This scarcity fundamentally constrains the ability of LLMs to learn proof patterns specific to the schedulability analysis domain.

The schedulability analysis lemmas are substantially more complex than those found in pure mathematics benchmarks. They are built upon a complex hierarchy of concepts, requiring many intricate definitions that often recursively depend on one another across multiple levels, along with numerous hypotheses. For example, in the popular PutnamBench suite [55] for evaluating LLM-based mathematics theorem proving, the description of a lemma spans no more than 20 lines. In contrast, the complete description of a schedulability analysis lemma in Prosa, including all definitions, variables, assumptions, and the lemma itself, often extends to several hundred lines. This increased length and conceptual depth impose a significantly heavier burden on the LLM’s context handling and reasoning capabilities. Beyond the complexity of individual lemmas, Prosa has a carefully designed, highly structured architecture that promotes lemma reuse across different task models. Whether an existing lemma can be applied to prove a target lemma depends on whether its preconditions hold, and those preconditions, in turn, rely on further preconditions, ultimately tracing back to the specific assumptions of the task model under consideration. This unique feature has introduced extra difficulty in the proving. We will further elaborate on this point in Section III-C.

To address the above challenges, our design adopts several complementary strategies. First, because pre-training alone cannot supply sufficient domain knowledge, we rely on strong inference-time scaling: we let the LLM repeatedly attempt the proof, using feedback from external tools. Second, we leverage the existing Prosa library to retrieve relevant definitions and lemmas, and to learn proving techniques by imitating how Prosa handles similar problems. This demands a powerful retrieval capability capable of accurately locating useful information within Prosa. Third, we incorporate optional human-provided proof hints, which help the agent navigate deep proof chains that would otherwise be difficult to resolve automatically.

For these considerations, we build ProsaBuddy on top of OpenCode [11] rather than constructing an agent system from scratch. OpenCode provides a solid foundation that satisfies the requirements outlined above. In OpenCode, an agent follows an iterative ReAct (Reasoning & Acting) loop [7]: it queries the LLM, uses the reasoning in the model’s output to execute external tools, and then appends the tool feedback to the conversation context for the next query, repeating this process until the task is resolved. OpenCode offers powerful code search capabilities well-suited for retrieval over the Prosa library and allows users to inject human-written proof hints into the agent’s prompt easily. By building on OpenCode, we inherit a mature agent foundation and can focus our efforts on developing high-level capabilities tailored to the needs of schedulability analysis lemma proving. Moreover, building ProsaBuddy on OpenCode makes it easy to use: users can obtain it by extending the original OpenCode with additional tools, prompts, and skills.

Figure 2 summarizes the major components of ProsaBuddy. The system takes as input a lemma formalized in Rocq, along with an *optional* human-written proof hints. The output is a verified proof in Rocq, or a failure declaration. For proof construction, ProsaBuddy uses LLM-based agents in a subgoal-delegation architecture. The primary agent decomposes the lemma into smaller, independent subgoals and dispatches them to specialized subagents. The subagent leverages OpenCode’s existing infrastructure, which already supports launching subagents. When launching a subagent, the system does not simply forward the original prompt. Instead, it builds a structured task bundle that includes the subgoal to prove, the necessary context, and a snapshot of the current proof state, for the subagent to solve the assigned subgoal independently. We detail the lemma decomposition process in Section III-B.

The agents retrieve relevant information from the Prosa codebase using OpenCode’s standard file-reading and code-search tools, such as file browsing and grep-like search. Beyond these standard capabilities, we enable ProsaBuddy to invoke Rocq tools directly to get rich feedback information for its ReAct loop. For each Rocq tool, we implement a dedicated skill that wraps its invocation and processes its output. We elaborate on the newly added external tools in Section III-D. Last but not least, we equip the agents with a collection of proving skills that target common challenges in real-time

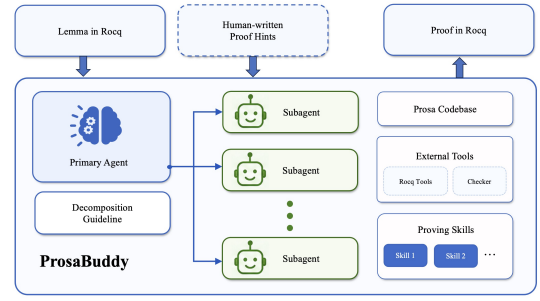


Fig. 2. Overview of ProsaBuddy

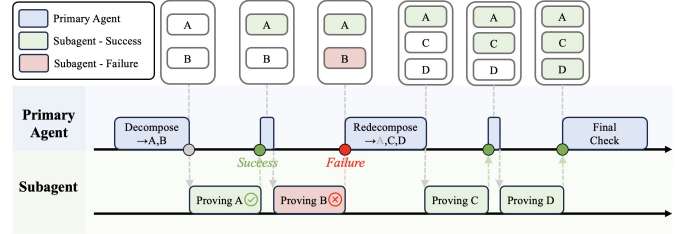


Fig. 3. An example of the decomposition-based proving procedure

schedulability analysis proofs. We elaborate on these proving skills in Section III-C.

B. Lemma Decomposition

The proving process starts with the primary agent, which first decides whether the lemma to be proved is simple enough to be proven by itself; if so, it attempts to complete the proof directly. If the lemma is complex, it performs lemma decomposition, i.e., breaks the target lemma into several subgoals. Each subgoal is then delegated to a subagent for proof. After launching a subagent, the primary agent suspends and waits for that subagent to return. **The subagent returns to the primary agent with either a success or a failure, accompanied by a failure report that explains why it failed and what important piece of information is missing for it to complete the proof of that subgoal.** Upon receiving a failure return from a subagent, the primary agent attempts to refine the decomposition. When the primary agent refines the decomposition, all subgoals that have already been proved will be kept. Figure 3 shows an illustrative example of the proving process. The primary agent first decomposes the target lemma into two subgoals, A and B. The subagent proving A returns with a success, but the one proving B returns with a failure. The primary agent then analyzes the failed subgoal, redo the decomposition, further divides subgoal B into smaller subgoals C and D, and dispatches them to subagents for proof. Once both C and D are successfully proved, the primary agent performs a final recheck to ensure that all generated proofs returned from the subagents compose into a valid proof of the original lemma.

The decomposition performed by the primary agent is mechanically checked for structural correctness: it guarantees that the original lemma follows from the conjunction of the subgoals. Concretely, the agent constructs a proof sketch in

TABLE II
SUMMARY OF DECOMPOSITION GUIDELINES

Name	Brief Description
Reshaping	If an expression is mathematically useful but its exact form does not match what a downstream lemma requires, we should create a subgoal dedicated to reshaping the expression into the exact needed form. Example: If the context contains $\text{size } (\text{filter } P \text{ } xs) \leq 1$, a useful subgoal is to reshape it into $\text{count } P \text{ } xs \leq 1$, so that a counting lemma can be applied directly.
Witnessing	An object may already be known to exist, but it is still hidden inside an existential statement or an informal paper description. Then the proof should create a subgoal to make that object explicit and record the facts that come with it before using it in later steps. Example: When the context contains an existential fact about a job arriving in an interval I and belonging to task tsk , a useful subgoal is to extract a concrete witness j together with facts such as $\text{arrives_in } j \text{ } I$ and $\text{job_task } j = tsk$.
Library Matching	Prosa, MathComp, or local lemma may already be the right tool, but the current proof state is not ready for applying it. Then the proof should create a subgoal dedicated to preparing that lemma application: specializing general hypotheses to the current case, and writing out the required premises as explicit subgoals to be proved. Example: Suppose a Prosa lemma can prove the desired conclusion once it is given premises such as $\text{job_task } j = tsk$ and $\text{arrives_in } j \text{ } I$. A useful low-level decomposition is to create separate subgoals for these premises, prove them first, and then apply the library lemma directly.
Case Splitting	A proof may need to be split into cases, but each branch should not be packed into one big proof region. Then the proof should create a subgoal to make the case split explicit: write out the branch condition, and leave the rest of the branch proof to later subgoals. Example: We already know that the next step depends on whether a job j belongs to the set of jobs arriving in interval I , but we should have a subgoal dedicated to splitting $j \in \text{jobs_arrived_during } I$, so that one branch explicitly carries the fact $j \in \text{jobs_arrived_during } I$, and the other branch explicitly carries the fact $j \notin \text{jobs_arrived_during } I$. Then the latter proof can handle the two branches separately, instead of hiding the whole case analysis inside one large proof step.
Lifting	When a goal concerns an set (e.g., counting, cardinality, or aggregation) but only per-element implications are available, introduce a subgoal to lift them to the set level. Example: If each job j that receives service in an interval I must have arrived before the end of I , a useful subgoal is to lift this fact to a counting inequality: $\text{count } (\text{fun } j \Rightarrow \text{receives_service_in } I \text{ } j) \text{ } \text{jobs} \leq \text{count } (\text{fun } j \Rightarrow \text{arrived_before } (\text{end_of } I) \text{ } j) \text{ } \text{jobs}$.

```

1  Lemma T (a b c d e : nat) :
2    a + c <= b + d.
3  Proof.
4    have H1 : a <= b.
5    {
6      have S1 : a <= e.
7      { admit. }
8      have S2 : e <= b.
9      { admit. }
10     exact: leq_trans S1 S2.
11   }
12   have H2 : c <= d.
13   { admit. }
14   exact: leq_add H1 H2.
15 Qed.

```

Fig. 4. An example of the proof script for lemma decomposition

which each subgoal is replaced by `admit`, and then verifies that the sketch is accepted by the Rocq compiler. Figure 4 gives an example of such a decomposition. Nested decomposition is permitted (e.g., introducing a `have` tactic within another `have`, provided it reflects a genuine sub-step or a structural refinement of the reasoning). However, intermediate subgoals (those that are further decomposed) are not delegated to subagents; only terminal subgoals (those that are *not* further decomposed) are delegated to subagents for proof. In the proof script presented in Figure 4, the original lemma `T` is decomposed into `H1` and `H2`, where `H1` is an intermediate subgoal and `H2` is terminal. `H1` is further decomposed into the terminal subgoals `S1`, `S2`, and `S3`. The terminal subgoals `S1`, `S2`, `S3`, and `H2` are then dispatched to subagents for proof.

The primary agent performs lemma decomposition in two stages. First, a high-level decomposition stage reasons about the current context (incorporating any human-written proof hint, if given) to extract fundamental steps that can genuinely advance the proof. Second, a low-level decomposition stage transforms those high-level steps into statements that are directly provable in Rocq. It turns out that the agent is doing

relatively well with the high-level decomposition, especially when human-written proof hints are given. However, the agent struggles in the second stage. Relying on the backbone LLM’s inherent reasoning capability, the low-level decomposition is usually not fine-grained enough. To solve this problem, we further augment the agent with extra guiding principles that are extracted manually from our experience in using Rocq for theorem proving. These principles provide instructions tailored to situations where low-level decomposition is effective (summarized in Table II). Specifically, they target highly localized sub-problems that are largely independent of the surrounding proof, making them ideal for a subagent to solve in a clean, isolated context. The guiding principles are embedded in the main agent’s prompt. The full prompt is considerably more detailed, and Table II gives only a brief overview.

C. Proving Skills

We develop proving skills to enhance the subagent’s ability to prove delegated subgoals. These skills are generated by using the LLM to analyze the proving traces, using the following process.

First, we select 65 lemmas from Prosa (only a small fraction of Prosa) as *training lemmas*. The selection criteria aim for diversity: we choose lemmas with proofs that vary in tactic count (i.e., difficulty), and we sample from different files and, whenever possible, different subdirectories within the Prosa codebase. The supplementary materials specify which lemmas are included in the training set.

We then use a preliminary version of our system (already equipped with external Rocq tools and the Prosa codebase) to attempt to prove these lemmas, and we collect the full traces of the proving procedure. Each trace captures the agent’s iterative attempts, including failures, error messages, and proof states, until success. We then fed these traces to an LLM to analyze failure reasons and generate skills. Concretely:

TABLE III
SUMMARY OF PROVING SKILLS

Name	Brief Description
Small-step & inspect	Write one tactic at a time and inspect the resulting goal state after each step. This avoids writing long proof scripts without checking intermediate goals, and it helps keep the actual proof state in view.
Error priority	Resolve errors in priority order: syntax or parse errors first, then ill-typed terms, then unsolved goals or tactic failures, since later diagnostics are often consequences of earlier failures.
Tactic selection	Use simpler and more direct tactics before heavier or more search-based ones, such as <code>reflexivity</code> , <code>lia</code> , <code>ring</code> , <code>tauto</code> , <code>congruence</code> , <code>auto/eauto</code> , and <code>decide/native_decide</code> . For composite goals, decompose the goal first and automate each component separately.
Rewrite discipline	When <code>rewrite</code> fails because the rewritten term appears in dependent types, first reorganize the context with commands like <code>generalize dependent</code> or <code>subst</code> . Also, name small helper results with <code>assert</code> , <code>pose proof</code> , or <code>set</code> instead of packing too many steps into one long tactic chain.
Cleanup & final check	After a proof compiles, clean it up, choose deliberately between <code>Qed</code> and <code>Defined</code> , and then actually compile the file with a command such as <code>coqc</code> to make sure everything still works. A proof is not complete until temporary holes such as <code>admit</code> or <code>Admitted</code> , have been removed.
Syntax-driven rewrite	Before each rewrite, check whether the pattern required by the lemma already appears exactly in the current goal. If it does not, first transform the goal into that form instead of forcing the rewrite just because it seems mathematically correct.
Local bridge lemmas	Use a small local bridge lemma when the mismatch is specific to the current branch or to a fixed local expression. Extract a generic lemma only when the same normalization gap recurs across proofs.
Minimal expansion	When a <code>by</code> line fails, remove only <code>by</code> and keep the rest of the tactic exactly the same. Re-run immediately and record both the first revealed error message and the number of focused goals. If expansion produces multiple goals, the primary issue is proof structure. If it reveals a rewrite mismatch, the issue is the current goal shape. If it reveals <code>Cannot apply</code> or <code>No assumption</code> , the issue lies in the lemma preconditions or the local context.
Expand “by” patterns	Expand compressed forms such as <code>by rewrite</code> , <code>by apply:</code> , <code>have ...by ...</code> , and <code>case ...; by ...</code> into ordinary proof scripts so that you can see exactly which step fails or which branch is still open. After expanding one <code>by</code> step, fix only that step. Do not guess new tactics before reading the first real error, and shorten it back to <code>by</code> only after the expanded version is confirmed to close the proof correctly.
Bool/Prop boundary	If the remaining reasoning is easier to carry out in <code>Prop</code> than in <code>Boolean</code> form, convert the comparison once and finish the branch in <code>Prop</code> , reflecting back only if the final goal is <code>Boolean</code> .
Normal form & bridge order	First decide what form the final result should have: a count expression or a big-operator expression. After choosing that target form, rewrite in one direction: <code>indicator sum -> filtered big operator -> count</code> . Do not switch back and forth between forms.
Big-op normalization order	First simplify the innermost sums that are easy to evaluate, such as constant sums or sums of <code>Boolean</code> tests. Then reorganize the outer sum if needed. Only after that should you rewrite repeated addition as multiplication. Try to keep the whole branch in one consistent form instead of repeatedly switching between sums, <code>iter</code> , and multiplication.
Count before min	First rewrite the main counting fact into a clear bound on <code>count</code> , such as <code>count P s <= k</code> . Only after that should you handle <code>Nat.min</code> , <code>minn</code> , or the final arithmetic inequality. Do not start the final arithmetic steps while the same quantity is still written in several different counting forms.
Failure signals	If <code>big_mkcond</code> or <code>sum1_count</code> does not apply, the expression is usually still in the wrong form. A common mistake is to skip the intermediate normalization steps and jump straight from an indicator sum to <code>count</code> or even to a <code>min</code> inequality, which makes both the rewrites and the later arithmetic harder to control.
Failure signatures	Failures of <code>When</code> rewrites such as <code>mulln</code> , <code>mulnC</code> , or <code>big_ord_rec</code> fail, the expression is usually still in the wrong form. If the branch has become a counting problem involving <code>count</code> , <code>sum1_count</code> , or <code>Nat.min</code> , switch to the count-normalization skill instead of continuing with generic big-operator rewrites.
Goal management	Use this skill when a <code>Coq</code> or <code>Ssreflect</code> proof has drifted into a multi-goal state after a branching tactic or local sublemma, especially when <code>coqc</code> reports “Expected a single focused goal but 2 goals are focused,” and the next step should be structural recovery with <code>bullets</code> , <code>braces</code> , and <code>explicit branch closure</code> rather than further mathematical reasoning. ¹
Goal-driven lemma instantiation	This skill helps distinguish between ordinary subgoals that can be proved later and early parameters or prerequisites that <code>Coq</code> must know immediately. It recommends using <code>apply</code> or <code>apply</code> to let the current goal expose the real remaining obligations, and, if necessary, supplying only the first missing parameter or preconditions instead of writing a long, fully explicit exact term.

¹ Goal-management errors were the most frequent failures in our experiments. This is likely because `Rocq` and newer `Ssreflect` proof modes enforce goal focusing and subproof bookkeeping more explicitly, so compressed proof scripts that worked in older `Coq` settings may fail in `Rocq`. We hypothesize that current LLMs have seen more older `Coq` scripts than `Rocq 9.x` scripts, and therefore often generate fragile goal-management patterns that lead to these errors.

- 1) We use LLM to divide each trace into segments, each handling a local task (e.g., fixing an error in the proof).
- 2) For each segment, we use the LLM to identify the problem being solved, analyze its cause, and generate a skill to avoid or resolve it. This produces a pool of proving skill candidates.
- 3) We use the LLM to merge proving skill candidates that address the same or similar problems, refining them to capture common patterns.
- 4) Finally, we manually review the remaining candidates, discarding those that are too general, vague, or lacking

useful information, and select the final 17 proving skills as summarized in Table III.

Due to space limitations, we cannot elaborate on every proving skill in Table III. In the following, we only give a detailed explanation of the *goal-driven lemma instantiation* proving skill (the last row in Table III), which addresses a problem specific to schedulability analysis using *Prosa*. As mentioned in Section III-A, *Prosa* has a highly structured hierarchy that facilitates lemma reuse across different task models. Applying such a lemma requires satisfying its preconditions, which themselves depend on further preconditions, eventually


```

1 exact (@uniprocessor_response_time_bound_restricted_supply_seq
2   Task H (@limited_preemptions_rtc_threshold Task H H1)
3   Job H2 H4 H3 (@limited_preemptive_job_model Job H5)
4   PState H_uniprocessor_proc_model H_unit_supply_proc_model
5   H_consumed_supply_proc_model arr_seq VALID_ARRIVALS
6   sched JOBS_FROM_ARRIVAL_SEQUENCE JOBS_MUST_ARRIVE
7   COMPLETED_JOBS_DONT_EXECUTE VALID_COSTS ts tsk H_tsk_in_ts
8   VALID_PREEMPTION_MODEL VALID_RTCT H0 VALID_ARRIVAL_CURVE
9   RESPECTS_MAX_ARRIVALS
10   (rs_jlfp_interference arr_seq sched)
11   (rs_jlfp_interfering_workload arr_seq sched)
12   ABSTRACT_WORK_CONSERVING H_sequential_tasks
13   I_AND_W_SEQUENTIAL
14   L_BUSY_INTERVALS_BOUNDED (arm_sbf  $\Pi$   $\Theta$  $\nu$ )
15   INTRA_BOUNDED R SOL_SEQ_RS
16   j ARR TSK).

1 apply : uniprocessor_response_time_bound_restricted_supply_seq;
2 try done.

```

Fig. 5. Example illustrating the goal-driven lemma instantiation proving skill. The upper part shows the proof script using `exact`, i.e., explicitly providing all the needed preconditions and parameters. The lower part shows the proof script using `apply` combined with `try done`.

reaching the specific assumptions of the chosen task model. In these cases, the agent tends to use the `exact` tactic when applying such lemmas, which requires all preconditions and parameters to be explicitly supplied. The upper part of Figure 5 shows a real example we take from the traces for generating the proving skills. In this example, the lemma requires a long list of preconditions and parameters (lines 2 - 16) if using `exact` to apply this lemma. It is very difficult for the LLM to precisely provide all the required preconditions and parameters. Even if the agent repeatedly guesses parameters, receives error messages from Rocq tools, and tries again, in most cases, it still can never find the correct combination. However, a simple solution exists for this problem: instead of using `exact` with all parameters, one can use the `apply` tactic with the lemma. This turns the preconditions and parameters into explicit subgoals, which are typically much easier to prove (many can be resolved automatically by built-in tactics). For instance, the lower part of Figure 5 shows the proof script that applies the lemma using `apply`, without providing the preconditions, and then using `try done` can close all the generated goals effortlessly. This insight becomes a reusable proving skill.

Why does analyzing proof traces enable the LLM to generate useful skills? The traces contain rich diagnostic information: they show iterative attempts to solve a local problem, with clear success/failure outcomes, error messages, and the current proof state at each step. The LLM can observe how the agent fails repeatedly and how it eventually succeeds. Even if the LLM does not fundamentally understand the domain, it can compare the correct and incorrect attempts and extract knowledge that was not originally part of its training. The resulting proving skill thus encodes a practical solution that the LLM alone would not produce.

It is worth comparing proving skills with the decomposition guidelines introduced in Section III-B. Both serve the same purpose: they add extra information to the LLM’s prompt to improve its problem-solving ability. The crucial differences

lie in how they are obtained and how they are used. First, decomposition guidelines are crafted manually by human experts, while proving skills are generated by having the LLM analyze proof traces, followed by human review. Actually, we also attempted to use a similar trace-driven process to extract decomposition guidelines, but the results were poor. We conjecture that this is because the impact of a decomposition is non-local and diffuse; the LLM struggles to connect later proof failures to earlier decomposition choices and to infer how the decomposition should be refined. Therefore, we rely on manual effort for decomposition guidance. Moreover, while the proving skills are stored in skill files and loaded on demand, decomposition guidelines are placed directly into the agent’s prompt. The current set of decomposition guidelines is small, so the benefit of making them load-on-demand is limited. Nonetheless, nothing prevents users from packaging new decomposition guidelines as on-demand skill files in the future.

D. External Tools

In addition to the existing tools in OpenCode—such as file reading and code search—we have integrated several specialized external tools. These include a suite of Rocq tools, which provide detailed feedback based on the current proof state, and a dedicated checker tool to verify the final correctness of generated proofs.

Rocq provides three interfaces for interacting with proof files, each of which we expose as an external tool for the agents. Rocq compiler is a batch compiler for .v files that checks whole files and returns compilation status, error locations, and error messages. Rocq toplevel is an interactive interface for sending individual commands; we use it as a tool that lets agents query library objects, lemma types, assumptions, and definitions. rocq-lsp is a language server that maintains a live view of a proof file, providing incremental diagnostics, current goals, local hypotheses, and document structure; we wrap it as a tool that supplies fine-grained proof-state feedback, enabling agents to repair proofs step by step instead of relying only on whole-file compilation errors.

Each tool is implemented as a module with three components: a call name exposed to the agent, a natural-language description telling the agent when to use it, and an input schema specifying the required arguments. Once registered in the agent’s tool registry, these descriptions and schemas become part of the agent’s available tool list. During proof construction, the agent does not execute Rocq commands or access the proof environment directly. Instead, it decides which registered tool to call and supplies the required arguments. The agent runtime then dispatches the call to the corresponding tool implementation. This offers advantages over letting the agent operate raw command lines: inputs are validated against the schema, outputs are parsed (e.g., error locations, goals, hypotheses) so the agent avoids fragile text parsing, and execution is confined to a small set of safe, allowed actions.

These Rocq tools only check the proof relative to the file that the agent submits. They are not sufficient to determine

whether the final generated proof is actually correct. The reason is that the agent may “cheat” during proof construction. For example, an agent might secretly change the theorem statement, weaken a definition, or add an unproven axiom. Such changes could still result in a file that passes compilation, yet they would not constitute a valid proof of the intended lemma. Even if we add strong constraints to the prompt, this behavior may still occur because prompting does not impose any hard constraints on the LLM’s output.

To ensure that the agent really proves the target lemma rather than bypassing it, we develop a Checker tool for proof integrity checking. Checker compares the proof file generated by the agent with the original modeling file to detect whether the agent has changed the problem itself, introduced unproven facts, weakened the theorem statement, or moved the main proof obligation into another global lemma. It treats edits outside the intended proof region as suspicious. Concretely, it reports cheating if the agent adds constructs such as `Axiom`, `Parameter`, `Parameters`, `Conjecture`, `Conjectures`, `Admitted`, `admit`, or `Abort`. It also detects changes that modify the target lemma statement, change existing model definitions, or introduce new global lemmas or definitions outside the proof region in a way that carries the main proof burden. These changes are considered cheating because the resulting file may compile without actually proving the original lemma. On the other hand, Checker allows harmless changes that do not alter the theorem’s meaning. It permits importing additional libraries from trusted sources such as Prosa or MathComp, adding local `Let` bindings inside the proof, and making purely cosmetic edits such as comments, blank lines, or formatting changes.

The Checker is implemented as an auditing script that runs once in the final phase. For the proof file under test, the script compares the final version with the original proof file to determine whether the agent has attempted to bypass the proof obligation. In our workflow, a lemma is considered successfully proved only when two conditions hold simultaneously. First, the generated `.v` file must pass the Rocq compiler check. Second, the checker must confirm that the proof does not rely on forbidden edits or semantic changes to the original modeling file. If either of them fails, the proof attempt is not counted as successful.

Beyond checking the generated proof file itself, we also run each proof task in a file-system sandbox. The sandbox is an isolated workspace that limits what files the agent can see and modify. This is necessary because, without such isolation, an agent could bypass the proof task by modifying files outside the intended workspace, such as installed libraries or dependency files. Such changes could alter the environment in which the theorem is checked, making the proof pass without actually proving the original theorem.

IV. EXPERIMENTS

A. Comparison with State-of-the-Art

We compare ProsaBuddy with two recent LLM-based automated theorem proving systems for Coq/Rocq: PALM [8] and

RocqStar [10]. PALM’s interaction layer with Coq relies on `coq-serapi`, which has only been maintained up to Coq 8.12. This dependency prevents PALM from being directly applied to the Rocq environment. Since our experiments use Rocq 9.0 for RocqStar and ProsaBuddy, we ported the subset of `coq-serapi` interaction functionality used by PALM to Rocq 9.0 for a fair comparison. We also attempted to evaluate COPRA [9], another recent LLM-based Coq/Rocq proving agent, but COPRA failed on all benchmark tasks in our environment; therefore, we do not include its results in Table V. We investigated the failures and found that, although COPRA claims support for both Lean and Coq, the released GitHub implementation leaves several Coq-environment connection functions unimplemented. We did not compare with Rango [36] because RocqStar has already included a direct comparison with Rango and reported better performance [10]. Therefore, we only choose the better one as the representative baseline in our experiments. We did not compare with CoqPilot because RocqStar is a follow-up system that extends the CoqPilot pipeline with a more advanced agentic architecture [10], [35]. We did not compare with ReCent-Prover [37] because its implementation is not yet publicly available. Moreover, we also include the original OpenCode in the comparison. For all systems, we use GPT-5.4 as the backbone LLM. All compared systems have access to the full Prosa library.

The evaluation is conducted on a benchmark that we created. The benchmark contains 24 lemma formalizations in Rocq, which are not included in the existing Prosa library. These lemmas are drawn from five research papers [56]–[59], [61] and one book [60] in real-time scheduling theory, and we manually formalized them in Rocq. This ensures that the benchmark we use has no overlap with the “training” lemmas used to automatically generate our proving skills, as introduced in Section III-C. The sources of these lemmas are listed in Table IV. All lemmas concern global multiprocessor scheduling. Note that, in Table IV, 10 and 11, as well as 12 and 13, come from manually splitting a single lemma into separate proof goals. We do this when the original lemma states multiple independent results at once. This split does not change the mathematical content of the source result; it only separates independent claims into smaller proof targets.

There is one subtle compatibility issue: Prosa 0.4 includes foundations for global multiprocessor scheduling analysis, but the latest version, Prosa 0.6, does not have this part. Prosa 0.6 is based on Rocq, whereas Prosa 0.4 is based on Coq, and Rocq and Coq have subtle syntactic differences. Therefore, we ported the global multiprocessor scheduling part in Prosa 0.4 from Coq to Rocq.

Our benchmark is constructed from existing papers in the field of real-time scheduling. In the original publications, each theorem or lemma is accompanied by a proof that was manually developed by human. For ProsaBuddy and OpenCode, we use the relevant portion of the original paper as the human-written proof hint input. Specifically, this input includes the target theorem or lemma itself, its full human-written proof, as well as any supporting definitions, notation, and variables that

TABLE IV
INFORMATION OF THE LEMMA IN OUR BENCHMARK

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Source	[56] Lem 3	[56] Lem 4	[56] Thm 6	[57] Thm 1	[57] Thm 2	[57] Thm 3	[57] Thm 4	[58] Lem 1	[58] Thm 1	[58] Lem 1-1	[58] Lem 1-2	[58] Lem 2-1	[58] Lem 2-2	[58] Lem 3	[58] Lem 4	[58] Lem 5	[58] Thm 1	[58] Thm 2	[59] Thm 3	[59] Lem 4	[59] Lem 5	[60] Lem 18.1	[60] Lem 18.6	[61] Lem 8
Dependency	-	1	1,2	-	-	3	-	-	8	-	-	10,11	10,11	12,13	14	-	15	16	21	-	20	-	22	-
# of Tactics	70-110	160-200	-	180-300	10-40	70-90	10-40	60-80	30-70	200-220	150-200	40-60	-	-	15-30	-	10-20	100-150	20-80	-	-	15-90	-	60-80

Note: “Dependency” represents which other lemma in this table are prerequisites for this lemma. “# of Tactics” represents the *range* of the number of tactics in the proof generated by different systems in the following evaluation experiments (different systems often generate proofs with different numbers of tactics). For those lemmas/theorems which are not proved by any system, the number is denoted by “-”.

are referenced within the proof. On the other hand, we do not supply such human-written proof hints to PALM or to RocqStar. The reason is that neither of these two systems exposes an input interface that can read external textual materials. PALM operates exclusively through its generate-and-repair workflow, which takes the formal theorem and the current proof state as inputs and iteratively attempts to construct a valid proof. It does not accept additional natural language hints or proof sketches from outside. Similarly, RocqStar is designed around formal Rocq context retrieval; it can query and retrieve relevant formal definitions and theorems from a structured library, but it lacks the ability to ingest free-text descriptions as external input. Later in the ablation study we will show that, even if the human-written proof hint is removed, ProsaBuddy still significantly outperforms PALM and RocqStar.

Table V shows the experiment results of the comparison. Each column in the table is the lemma index in the benchmark. In each cell, ✓ represents that the lemma is successfully proved, while ✗ the agent declares a failure to prove this lemma and stops. The number under ✓ represents the total amount of token consumption proving this lemma in the unit of 1K tokens (counting both input tokens and output tokens). ∞ represents that the agent has not yet proved the lemma before reaching the timeout threshold. In our experiments, we set the timeout threshold to be 50,000K tokens. The timeout is measured by the number of tokens consumed rather than by actual time consumption, since variations in time delay depend heavily on many factors unrelated to the model inference, such as network latency, server load, or local hardware restrictions. In contrast, the token count provides a more stable metric that better reflects the amount of LLM inference performed. To give readers a rough sense of scale, in most cases our method (ProsaBuddy) takes between 3 and 5 hours to reach the 50,000K-token threshold.

To reduce time and cost, we omit some experiments, which are denoted by “-” in Table V. This is because, as ProsaBuddy has shown much better performance than all the compared systems, we infer that for the cases where ProsaBuddy failed, all the other compared system will also very likely fail, so we omit these experiments to save time and cost.

From the experiment results, we can see that ProsaBuddy significantly outperforms the baselines. Out of 24 lemmas, PALM only succeeds 1 lemma, both RocqStar and OpenCode succeed 3, while ProsaBuddy succeed 17. For the one lemma PALM succeeds, PALM uses many fewer tokens. This is be-

cause of the way PALM works: In its workflow, PALM invokes the LLM exactly once to generate a complete initial proof. Following this single-shot generation, PALM does not interact with the LLM further, while detailed error corrections rely on automated proof generation tools such as CoqHammer [24]. As a result, the token consumption during the error correction stage is zero. This differs from other agent-based approaches, which typically require iterative, multi-turn dialogues with the LLM for both proof generation and error correction.

Our benchmark targets lemmas that are not yet included in Prosa but are closely connected to the foundations already provided by Prosa. Their proofs may rely on existing Prosa definitions and basic lemmas. This selection is intentional: lemmas that are largely unrelated to the current Prosa foundation would require ProsaBuddy to invent new definitions and many intermediate lemmas from scratch, which is beyond its current capability and outside the intended use case. Accordingly, the experimental results should not be interpreted as showing that ProsaBuddy can prove real-time scheduling theorems in full generality. They instead evaluate whether ProsaBuddy can extend an existing mechanized library by proving new lemmas when the relevant definitions and many auxiliary results are already available. This reflects the positive feedback loop we aim to support: as Prosa grows to cover more real-time scheduling topics, ProsaBuddy can benefit from the richer foundation and help prove further lemmas.

We further conduct evaluations using additional lemmas selected from Prosa. These lemmas are categorized into seven levels (L1–L7) based on proof length, measured by the number of tactics used in the proof. Each level contains 10 lemmas, with the exception of L7: Prosa includes only 12 L7 lemmas in total, among which we selected 5 as training lemmas and the remaining 5 as evaluation lemmas. All lemmas used in this evaluation are strictly distinct from the training lemma set employed in Section III-C.

This experiment evaluates whether ProsaBuddy can complete proofs that follow the native style, definitions, and proof patterns of Prosa itself. This allows us to test the system’s ability to work inside an existing mechanized development, rather than only on externally constructed case studies.

Unlike the our build benchmark, this experiment does not provide the agent with human-written proof hints. This is because many lemmas in Prosa are library-internal results and do not have corresponding human-written proofs in the literature.

TABLE V
RESULTS OF THE COMPARISON EXPERIMENTS AND ABLATION STUDY WITH OUR BUILT BENCHMARK

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	# of success
PALM	✗	✗	-	✗	✓ ₃	✗	✗	✗	✗	✗	✗	✗	-	-	✗	-	✗	✗	✗	-	-	✗	-	✗	1
RocqStar	✗	✗	-	✗	✓ ₂₃	✗	✗	✗	✗	✗	✗	✗	-	-	✗	-	✓ ₅₈₇	✗	✓ _{1,486}	-	-	✗	-	✗	3
OpenCode	✗	✗	-	✗	✓ ₂₄₇	✗	✗	✗	✓ _{1,083}	✗	✗	✗	-	-	✗	-	✓ ₅₀₀	✗	✗	-	-	✗	-	✗	3
PB-w/o-S	✓ _{25,410}	∞	-	✓ _{23,440}	✓ ₂₀₆	✓ _{3,760}	∞	✓ _{21,540}	✓ ₅₃₅	✗	✓ _{45,185}	∞	-	-	✓ _{4,084}	-	✓ ₆₀₁	✓ _{2,398}	✓ ₈₉₆	-	-	✓ _{29,129}	-	✗	12
PB-w/o-D	✓ _{23,056}	✓ _{20,858}	-	✓ _{11,296}	✓ ₁₈₆	✓ _{11,481}	✓ _{2,617}	✗	✓ ₄₅₆	∞	✓ _{30,334}	∞	-	-	✓ _{11,095}	-	✓ ₆₁₆	✓ _{8,798}	✓ ₄₇₉	-	-	✓ _{8,680}	-	✓ _{29,137}	14
PB-w/o-P	✓ _{37,125}	✗	-	✗	✓ ₂₀₅	✓ _{30,139}	✓ _{47,333}	✗	✓ _{6,511}	✓ _{35,156}	✓ _{24,619}	✗	-	-	✓ _{3,317}	-	✓ _{2,177}	✓ _{9,354}	✓ _{1,625}	-	-	✓ _{27,604}	-	✗	12
PB-w/o-H	✓ _{36,340}	∞	-	✓ _{11,238}	✓ ₃₅₈	✓ _{3,501}	✓ _{1,257}	✓ _{16,550}	✓ _{1,398}	✓ _{41,562}	✓ _{31,716}	✗	-	-	✓ _{23,906}	-	✓ _{1,321}	✓ _{5,096}	✓ _{1,833}	-	-	✓ _{21,157}	-	✓ _{9,278}	15
ProsaBuddy	✓ _{48,819}	✓ _{42,027}	∞	✓ _{13,332}	✓ ₂₈₁	✓ _{1,295}	✓ _{2,077}	✓ _{13,077}	✓ _{1,888}	✓ _{24,330}	✓ _{36,004}	✓ _{38,749}	✗	✗	✓ _{6,736}	✗	✓ _{3,908}	✓ _{2,229}	✓ ₃₀₇	✗	∞	✓ _{31,614}	✗	✓ _{13,557}	17

Note: The number under each ✓ represents the amount of token consumption to prove this lemma, which is in thousands of tokens.

TABLE VI
RESULTS WITH LEMMAS SELECTED FROM PROSA

	L1	L2	L3	L4	L5	L6	L7
# of tactics	1-20	21-40	41-60	61-80	81-100	101-150	> 150
PALM	1/10	0/10	-	-	-	-	-
RocqStar	7/10	2/10	-	-	-	-	-
OpenCode	4/10	0/10	-	-	-	-	-
ProsaBuddy	10/10	10/10	8/10	9/10	9/10	6/10	1/5

Note: “# of tactics” represent the number of tactics used in its proof.

We observe that Prosa contains many families of highly similar proofs. For example, the same lemma is often established under several related scheduling models, such as fixed-priority preemptive, non-preemptive, and limited-preemptive scheduling. Although these are technically different lemmas, their statements and proof structures can be very similar. In such cases, an agent with access to the full library may solve the target lemma largely by copying or lightly adapting an existing proof, rather than by generating the proof from the target statement and local context. This would make the evaluation less interesting. To solve this problem, for each benchmark lemma, we provide only the minimal Prosa slice required to compile the target file, computed from the transitive dependency closure of its required modules. The runner first computes the set of Prosa files that are necessary to compile the target file, including both direct and indirect dependencies. It then copies only these required files into the local workspace and compiles this reduced library. In this way, the agent has enough formal context to prove the benchmark lemma, but it cannot search unrelated parts of Prosa for the original proof or a near-duplicate proof.

Similar to above experiments, we omit the experiments with lemmas in L3-L7 for the three compared systems, which are denoted by “-” in Table VI. This is because, if a baseline shows little or no ability to solve lemmas in lower-level groups, there is no strong reason to expect better performance on higher levels. As PALM, RocqStar, and OpenCode solved zero or only a very small number of lemmas at L2. Therefore, we did not further evaluate these baselines on L3 and above.

The experiment results with lemmas selected from Prosa confirm the superiority ProsaBuddy over all the competing systems. The results also shows that ProsaBuddy still can successfully prove non-trivial lemmas even without human-written proof hints.

B. Ablation Study

We conduct ablation experiments to evaluate the contribution of different components in ProsaBuddy, the result of which is also reported in Table V.

PB-w/o-S. This variant removes the proving skills from ProsaBuddy. It proves 12 lemmas, fewer than the full ProsaBuddy, demonstrating the effectiveness of the proving skills developed in Section III-C.

PB-w/o-D. This variant removes the lemma decomposition mechanism using subagents. Instead, it uses only the primary agent in OpenCode to prove a lemma. PB-w/o-D proves 14 lemmas, fewer than the full ProsaBuddy, which shows the effectiveness of the lemma decomposition mechanism in Section III-B. The gap between PB-w/o-D and ProsaBuddy mainly appears on harder cases (i.e., the lemmas ProsaBuddy consume more tokens to prove). The token numbers show a trade-off: ProsaBuddy sometimes spends more tokens because planning and delegation introduce extra interaction cost, but this overhead enables proofs that the single-agent variant cannot complete. For simpler lemmas, PB-w/o-D often uses comparable or fewer tokens, suggesting that the lemma decomposition mechanism is useful for difficult lemmas rather than for every proof.

PB-w/o-H. This variant removes the human-written proof hints from ProsaBuddy, i.e., it does not read the corresponding paper content. PB-w/o-H proves 15 lemmas. The 2 cases where ProsaBuddy succeeds but PB-w/o-H does not are both relatively more difficult lemmas. This suggest that the for simple cases, the formal context in Prosa, including definitions, existing lemmas, and proof states, already provides substantial guidance for proof construction. However, for difficult lemmas, the human-written proof hints plays a more significant role. Moreover, by comparing **PB-w/o-H** with PALM and RocqStar, which also does not have human-written proof hints,

TABLE VII
EXPERIMENT RESULTS WITH DIFFERENT BACKBONE LLMs

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	# of success
GPT-5.4	✓	✓	∞	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✓	✗	✓	✓	✓	✗	✗	✓	✗	✓	17
DeepSeek-v4-flash	∞	∞	-	∞	✓	✓	✓	✗	✓	∞	∞	∞	-	-	6,736	3,908	2,229	✓	307	-	-	✓	-	∞	9
DeepSeek-v4-pro	∞	∞	-	∞	2,201	∞	8,749	✗	4,247	✗	✗	✗	-	-	6,205	-	2,262	18,405	5,624	-	-	20,245	-	-	6
Gemini-3.1-flash-lite	✗	∞	-	∞	1,889	∞	∞	∞	✗	∞	✗	✗	-	-	20,898	-	768	✗	4,567	-	-	✗	-	∞	2
GPT-oss-120B	✗	✗	-	✗	1,192	✗	✗	✗	✗	✗	✗	✗	-	-	✗	-	✗	✗	10,355	-	-	✗	-	✗	1

we can see that our design still significantly outperforms the state-of-the-art.

PB-w/o-P. This variant removes access to the full Prosa library and only provides the minimal dependent files needed to state the target lemmas. As a result, it can use basic definitions and low-level lemmas (related to the basic properties of the definitions) included in the dependent files, but cannot directly observe or reuse higher-level auxiliary lemmas in Prosa. PB-w/o-P proves 12 lemmas, fewer than ProsaBuddy. This shows that a full library context is important for extending an existing mechanized development. Without it, the agent often needs to rediscover intermediate facts that are already available elsewhere in Prosa or reconstruct them from lower-level definitions. The token numbers reflect this extra burden on some successful cases.

C. Different Backbone LLMs

We further evaluate how the choice of difference backbone LLM affects the performance of ProsaBuddy. The results are reported in Table VII. In these experiments, all components of ProsaBuddy are held constant except the backbone LLM. From the results we can see that the backbone LLM has a substantial impact on proof success: with the most powerful model, GPT-5.4, ProsaBuddy proves 17 lemmas, compared to 9 with DeepSeek-v4-flash, 6 with DeepSeek-v4-pro, 2 with Gemini-3.1-flash-lite, and 1 with GPT-oss-120B. Unexpectedly, DeepSeek-v4-flash outperforms DeepSeek-v4-pro. We currently do not have a clear explanation for this anomaly and will investigate its cause in future work. Overall, these results show that the agent framework, augmented external knowledge, and feedback from Rocq tools are not sufficient on their own; the reasoning capability of the backbone LLM is crucial for the performance of ProsaBuddy. We also observe that, on the subset of lemmas the weaker models can prove, they tend to consume more tokens on average than GPT-5.4, with a few exceptions. Therefore, from an economic perspective, using a more powerful model to prove a lemma does not necessarily lead to a higher overall cost, even if its per-token price is higher. Similar to above, the mark “-” in Table VII denotes the omitted experiments to save time and cost, as for the difficult cases GPT-5.4 failed, the other weaker LLMs likely will also fail.

V. DISCUSSIONS AND CONCLUSIONS

In this work, we present ProsaBuddy, an LLM-based agent system designed to lower the effort needed to develop mechanized real-time schedulability proofs. Experimental evaluation shows that ProsaBuddy can successfully prove many non-trivial lemmas. Although in this paper we introduce and evaluate ProsaBuddy as a standalone system that proves lemmas end-to-end, our ultimate goal is to build ProsaBuddy as a highly interactive tool. When a human expert intervenes in the proving process and provides guidance at points where ProsaBuddy gets stuck, ProsaBuddy becomes significantly more effective and useful. We did not systematically evaluate ProsaBuddy with a human in the loop, because designing an objective and reproducible evaluation methodology for such interaction is difficult. Nevertheless, we conducted some ad-hoc interactive experiments. For example, for cases 13 and 14 in Table V, which ProsaBuddy could not prove alone, a researcher with expertise in real-time scheduling theory worked interactively with ProsaBuddy, injecting additional human instructions when ProsaBuddy was stuck or actively requested support. In this way, both lemmas were successfully proved in 2 hours and 3 hours, respectively. For comparison, a second researcher with a similar background independently proved the same two lemmas fully manually, requiring 14 hours and 17 hours, respectively.

Another critical issue in automated theorem proving is the formalization of the lemmas themselves. This is especially important for schedulability lemmas, whose descriptions are lengthy and involve many definitions and hypotheses. Correct formalization is the foundation for all subsequent proof work: if a lemma is incorrectly formalized, even a flawless proof is meaningless. Therefore, before any proof can be attempted, the lemma must be formalized very carefully—a task that itself demands considerable effort (though generally much less than that required for proving it). Assisting researchers during this formalization phase is thus also valuable. However, while lemma proving benefits from an automatic verifier (Rocq) that can check correctness, formalization lacks such a verifier. It is impossible to fully automate the checking of whether a formalization precisely captures the human’s intended meaning. In future work, we will investigate how to utilize an LLM-based agent to assist the formalization of schedulability analysis lemmas.

REFERENCES

- [1] K. Bletsas, N. C. Audsley, W.-H. Huang, J.-J. Chen, and G. Nelissen, “Errata for three papers (2004-05) on fixed-priority scheduling with self-suspensions,” *Leibniz Transactions on Embedded Systems*, vol. 5, no. 1, pp. 02–1, 2018.
- [2] M. Günzel and J.-J. Chen, “Correspondence article: Counterexample for suspension-aware schedulability analysis of edf scheduling,” *Real-Time Systems*, vol. 56, no. 4, p. 490–493, Oct. 2020.
- [3] J.-J. Chen, G. Nelissen, W.-H. Huang, M. Yang, B. Brandenburg, K. Bletsas, C. Liu, P. Richard, F. Ridouard, N. Audsley, R. Rajkumar, D. Niz, and G. Brüggem, “Many suspensions, many problems: a review of self-suspending tasks in real-time systems,” *Real-Time Systems*, vol. 55, no. 1, p. 144–207, Jan. 2019.
- [4] K.-H. Chen, M. Günzel, G. von der Brüggem, and J.-J. Chen, “Critical instant for probabilistic timing guarantees: Refuted and revisited,” in *2022 IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 145–157.
- [5] F. Cerqueira, F. Stutz, and B. B. Brandenburg, “Prosa: A case for readable mechanized schedulability analysis,” in *28th Euromicro Conference on Real-Time Systems (ECRTS 2016)*, 2016, pp. 273–284.
- [6] “The coq proof assistant,” project web site, <https://coq.inria.fr>.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] M. Lu, B. Delaware, and T. Zhang, “Proof automation with large language models,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, p. 1509–1520.
- [9] A. Thakur, G. Tsoukalas, Y. Wen, J. Xin, and S. Chaudhuri, “An in-context learning agent for formal theorem-proving,” in *First Conference on Language Modeling*, 2024.
- [10] A. Kozyrev, N. Khranov, G. Solovov, and A. Podkopaev, “Rocqstar: Leveraging similarity-driven retrieval and agentic systems for rocq generation,” in *The 25th International Conference on Autonomous Agents and Multi-Agent Systems*, 2025.
- [11] Anomaly, “OpenCode: The open source ai coding agent,” 2026, available at: <https://opencode.ai>; accessed May 25, 2026.
- [12] The Rocq Development Team, *The Rocq Prover Reference Manual*, Inria, CNRS, and contributors, 2025, version 9.2. [Online]. Available: <https://rocq-prover.org/refman/>
- [13] G. Gonthier, A. Mahboubi, and E. Tassi, “A Small Scale Reflection Extension for the Coq system,” Inria Saclay Ile de France, Research Report RR-6455, 2016.
- [14] F. Cerqueira, G. Nelissen, and B. B. Brandenburg, “On strong and weak sustainability, with an application to self-suspending real-time tasks,” in *30th Euromicro Conference on Real-Time Systems (ECRTS 2018)*, 2018, pp. 26–1.
- [15] S. Bozhko and B. B. Brandenburg, “Abstract Response-Time Analysis: A Formal Foundation for the Busy-Window Principle,” in *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*, 2020, pp. 1–24.
- [16] K. Bedarkar, M. Vardishvili, S. Bozhko, M. Maida, and B. B. Brandenburg, “From intuition to coq: A case study in verified response-time analysis 1 of fifo scheduling,” in *2022 IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 197–210.
- [17] P. Roux, S. Quinton, and M. Boyer, “A Between Response Time Analysis and Network Calculus,” in *34th Euromicro Conference on Real-Time Systems (ECRTS 2022)*, vol. 231, 2022, pp. 5:1–5:22.
- [18] M. Maida, S. Bozhko, and B. Brandenburg, “Foundational response-time analysis as explainable evidence of timeliness,” in *34th Euromicro Conference on Real-Time Systems (ECRTS 2022)*, 2022, pp. 1–25.
- [19] K. Bedarkar, L. Elbeheiry, M. Sammler, L. Gähler, B. Brandenburg, D. Dreyer, and D. Garg, “Refinedprosa: Connecting response-time analysis with c verification for interrupt-free schedulers,” *Proceedings of the ACM on Programming Languages*, vol. 9, pp. 73 – 97, 2025.
- [20] A. Mahboubi and E. Tassi, *Mathematical Components*. Zenodo, Sep. 2022.
- [21] M. Dénès, A. Mörtberg, and V. Siles, “A refinement-based approach to computational algebra in COQ,” in *Interactive Theorem Proving*, ser. Interactive Theorem Proving, L. Beringer and A. Felty, Eds., vol. 7406. Princeton, United States: Springer, Aug. 2012, pp. 83–98.
- [22] B. Ekici, A. Mebsout, C. Tinelli, C. Keller, G. Katz, A. Reynolds, and C. Barrett, “Smtcoq: A plug-in for integrating smt solvers into coq,” in *Computer Aided Verification*, R. Majumdar and V. Kunčák, Eds. Cham: Springer International Publishing, 2017, pp. 126–133.
- [23] B. Ekici, G. Katz, C. Keller, A. Mebsout, A. J. Reynolds, and C. Tinelli, “Extending SMTCoq, a Certified Checker for SMT (Extended Abstract),” in *EPTCS*, ser. EPTCS, J. C. Blanchette and C. Kaliszyk, Eds., vol. 210, Coimbra, Portugal, Jul. 2016.
- [24] z. Czajka and C. Kaliszyk, “Hammer for coq: Automation for dependent type theory,” *J. Autom. Reason.*, vol. 61, no. 1–4, p. 423–453, Jun. 2018.
- [25] D. Delahaye, “A Tactic Language for the System Coq,” in *Logic for Programming and Automated Reasoning (LPAR)*, ser. Lecture Notes in Computer Science (LNCS)/Lecture Notes in Artificial Intelligence (LNAI), M. Parigot and A. Voronkov, Eds., vol. 1955. Reunion Island (France): Springer, Nov. 2000, pp. 85–95.
- [26] B. Ziliani, D. Dreyer, N. R. Krishnaswami, A. Nanevski, and V. Vafeiadis, “Mtac: A monad for typed tactic programming in coq,” *Journal of Functional Programming*, vol. 25, p. e12, 2015.
- [27] J.-O. Kaiser, B. Ziliani, R. Krebbers, Y. Régis-Gianas, and D. Dreyer, “Mtac2: typed tactics for backward reasoning in coq,” *Proc. ACM Program. Lang.*, vol. 2, no. ICFP, Jul. 2018.
- [28] E. Tassi, “Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog dialect),” in *The Fourth International Workshop on Coq for Programming Languages*, Los Angeles (CA), United States, Jan. 2018.
- [29] M. Sozeau, S. Boulrier, Y. Forster, N. Tabareau, and T. Winterhalter, “Coq coq correct! verification of type checking and erasure for coq, in coq,” *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019.
- [30] E. Komendantskaya, J. Heras, and G. Grov, “Machine learning in proof general: Interfacing interfaces,” *Electronic Proceedings in Theoretical Computer Science*, vol. 118, p. 15–41, 2013.
- [31] K. Yang and J. Deng, “Learning to prove theorems via interacting with proof assistants,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6984–6994.
- [32] L. Blaauwbroek, J. Urban, and H. Geuvers, “The tactician,” in *Intelligent Computer Mathematics*, C. Benz Müller and B. Miller, Eds. Cham: Springer International Publishing, 2020, pp. 271–277.
- [33] A. Sanchez-Stern, Y. Alhessi, L. Saul, and S. Lerner, “Generating correctness proofs with neural networks,” in *Proceedings of the 4th ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, ser. MAPL 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 1–10.
- [34] E. First, Y. Brun, and A. Guha, “Tactok: semantics-aware proof synthesis,” *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020.
- [35] A. Kozyrev, G. Solovov, N. Khranov, and A. Podkopaev, “CoqPilot, a plugin for LLM-based generation of proofs,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, Oct. 2024, pp. 2382–2385.
- [36] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. a. F. Ferreira, S. Lerner, and E. First, “Rango: Adaptive retrieval-augmented proving for automated software verification,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025, p. 347–359.
- [37] Y. Sun, C. Shi, H. Lyu, and Y. Xiong, “On reasoning-centric llm-based automated theorem proving,” *arXiv preprint arXiv:2604.19558*, 2026.
- [38] Y. Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, M. Jamnik, and C. Szegedy, “Autoformalization with large language models,” *Advances in neural information processing systems*, vol. 35, pp. 32 353–32 368, 2022.
- [39] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample, “Draft, Sketch, and Prove: Guiding formal theorem provers with informal proofs,” in *International Conference on Learning Representations*, 2023.
- [40] Q. Liu, X. Zheng, X. Lu, Q. Cao, and J. Yan, “Rethinking and improving autoformalization: Towards a faithful metric and a dependency retrieval-based approach,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [41] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar, “Leandjo: Theorem proving with retrieval-augmented language models,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 21 573–21 612.
- [42] Z. Azerbayev, H. Schoelkopf, K. Paster, M. D. Santos, S. M. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck, “Llemma: An

open language model for mathematics,” in *The Twelfth International Conference on Learning Representations*, 2024.

- [43] H. Wang, H. Xin, C. Zheng, Z. Liu, Q. Cao, Y. Huang, J. Xiong, H. Shi, E. Xie, J. Yin, Z. Li, and X. Liang, “LEGO-prover: Neural theorem proving with growing libraries,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [44] H. Xin, Z. Ren, J. Song, Z. Shao, W. Zhao, H. Wang, B. Liu, L. Zhang, X. Lu, Q. Du, W. Gao, H. Zhang, Q. Zhu, D. Yang, Z. Gou, Z. Wu, F. Luo, and C. Ruan, “Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [45] H. Lin, Z. Sun, S. Welleck, and Y. Yang, “Lean-STar: Learning to interleave thinking and proving,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [46] P. Song, K. Yang, and A. Anandkumar, “Lean copilot: Large language models as copilots for theorem proving in lean,” in *Proceedings of the International Conference on Neuro-symbolic Systems*, ser. Proceedings of Machine Learning Research, G. Pappas, P. Ravikumar, and S. A. Seshia, Eds., vol. 288. PMLR, 28–30 May 2025, pp. 144–169.
- [47] Y. Lin, S. Tang, B. Lyu, Z. Yang, J.-H. Chung, H. Zhao, L. Jiang, Y. Geng, J. Ge, J. Sun, J. Wu, J. Gesi, X. Lu, D. Acuna, K. Yang, H. Lin, Y. Choi, D. Chen, S. Arora, and C. Jin, “Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [48] S. Varambally, T. Voice, Y. Sun, Z. Chen, R. Yu, and K. Ye, “Hilbert: Recursively building formal proofs with informal reasoning,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [49] A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygóźdź, P. Miłoś, Y. Wu, and M. Jamnik, “Thor: Wielding hammers to integrate language models and automated theorem provers,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 8360–8373.
- [50] E. First, M. N. Rabe, T. Ringer, and Y. Brun, “Baldur: Whole-proof generation and repair with large language models,” *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023.
- [51] H. Wang, H. Xin, Z. Liu, W. Li, Y. Huang, J. Lu, Z. YANG, J. Tang, J. Yin, Z. Li, and X. Liang, “Proving theorems recursively,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [52] K. Weng, L. Du, S. Li, W. Lu, H. Sun, H. Liu, and T. Zhang, “Autoformalization in the era of large language models: A survey,” 2025.
- [53] N. Giessing, H. Rahkooy, and M. Schubotz, “A survey on llms in automated theorem proving,” 2026.
- [54] “The lean mathematical library,” in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, ser. CPP 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 367–381.
- [55] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri, “Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition,” in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [56] M. Bertogna, M. Cirinei, and G. Lipari, “Improved schedulability analysis of edf on multiprocessor platforms,” in *17th Euromicro Conference on Real-Time Systems (ECRTS’05)*. IEEE, 2005, pp. 209–218.
- [57] M. Bertogna and M. Cirinei, “Response-time analysis for globally scheduled symmetric multiprocessor platforms,” in *28th IEEE International Real-Time Systems Symposium (RTSS 2007)*. IEEE, 2007, pp. 149–160.
- [58] N. Guan, M. Stigge, W. Yi, and G. Yu, “New response time bounds for fixed priority multiprocessor scheduling,” in *2009 30th IEEE Real-Time Systems Symposium*. IEEE, 2009, pp. 387–397.
- [59] Y. Sun, G. Lipari, N. Guan, and W. Yi, “Improving the response time analysis of global fixed-priority multiprocessor scheduling,” in *2014 IEEE 20th International Conference on Embedded and Real-Time Computing Systems and Applications*. IEEE, 2014, pp. 1–9.
- [60] S. Baruah, M. Bertogna, and G. Buttazzo, *Multiprocessor scheduling for real-time systems*. Springer, 2015.
- [61] A. Gujarati, F. Cerqueira, and B. B. Brandenburg, “Outstanding paper award: Schedulability analysis of the linux push and pull scheduler with arbitrary processor affinities,” in *25th Euromicro Conference on Real-Time Systems (ECRTS 2013)*, 2013, pp. 69–79.